PIMPRI CHINCHWAD COLLEGE OF ENGINEERING (PCCOE), PUNE

Department of Information Technology | Academic Year 2025-26

Object-Oriented Programming (OOP) Mini Project Report

Exam Timetable & Seating Generator

Automated, Clash-Free Examination Scheduling & Malpractice-Resistant Seating in Core Java

---

A comprehensive Object-Oriented software project demonstrating Abstraction, Inheritance, Polymorphism, Encapsulation, Design Patterns, Exception Handling, Graph Colouring, and Character Streams File I/O.

Class: S.Y. B.Tech (IT)

Course: Object Oriented Programming

Platform: Core Java (JDK 17+)

GitHub Repository: github.com/amritakharasne62006-design/oops_project-

Status: Tested & Fully Verified

1. Abstract & Executive Summary

In academic institutions, the scheduling of semester examinations and the physical seating allocation of examinees is predominantly handled through manual entry in spreadsheets. This manual workflow routinely results in student timetable clashes (e.g., overlapping backlog and regular courses), malpractice risks due to adjacent seating of same-subject students, unfair invigilation duty distributions, and hall capacity mismanagement.

The **Exam Timetable & Seating Generator** is an automated desktop system developed in pure **Core Java (JDK 17+)** that solves these challenges using classic Object-Oriented design principles and computer science algorithms:

- **Clash-Free Scheduling (Graph Colouring):** Models courses as vertices and common student enrollments as conflict edges, applying greedy graph colouring to allocate non-overlapping time slots.
- **Anti-Malpractice Seating (Strategy Pattern):** Implements interchangeable seating algorithms (Mixed-Subject Interleaved, Zig-Zag Snake, Sequential, Random) ensuring adjacent students never take the same exam paper.
- **Fair Invigilation (PriorityQueue):** Distributes invigilation duties evenly across faculty members using a min-heap ordered by total duties assigned.
- **Validation & Custom Exceptions:** Enforces hard constraints (zero student clashes, room capacity limits, holiday exclusions) and soft constraint warnings.
- **Reports & Character Stream I/O:** Generates master timetables, student hall tickets by PRN, ASCII room seat grids, and exports formatted CSV files using `BufferedReader` and `BufferedWriter`.

2. Problem Statement & System Objectives

Manual Spreadsheet Flaw	Automated Java Solution in This Project
Student Clashes: Simultaneous scheduling of two exams for a student enrolled in both.	**Graph Colouring Algorithm:** Detects common enrollments and mathematically enforces distinct time slots.
Malpractice Risk: Students of the same subject sitting side-by-side or behind each other.	**Mixed-Subject Interleaved Seating:** Desks alternate between different courses across rows and columns.
Poor Room Utilization: Booking an auditorium for 10 students or overcrowding small rooms.	**Capacity-Sorted Allocation:** Fills largest rooms first and enforces hard capacity bounds.
Unfair Faculty Workload: Unequal assignment of invigilation duties across teachers.	**Min-Heap PriorityQueue:** Always assigns duties to the available faculty member with fewest duties so far.

3. System Architecture & Class Hierarchy

The system is organized into modular packages under `com.examscheduler`:

```
com.examscheduler
├── model/        : Person (abstract), Student, Faculty, Course, Room, TimeSlot, Exam, SeatAllocation, In
├── strategy/     : SeatingStrategy (interface), MixedSubjectSeating, ZigZagSeating, SequentialSeating, R
├── service/      : TimetableGenerator (Graph Colouring), SeatingService, InvigilatorService (PriorityQue
├── validation/   : ConflictChecker (Hard and soft constraint auditor)
├── exception/    : SchedulingClashException, RoomCapacityExceededException, InsufficientInvigilatorsExce
├── report/       : ReportGenerator (Master timetable, hall ticket, seating chart, duty roster, statistic
├── util/         : FileHandler (FileReader, BufferedReader, FileWriter, BufferedWriter)
└── Main.java     : Interactive CLI console application entry point
```

UML Inheritance & Strategy Model

`abstract class Person` → Extended by `Student` (adds PRN, courses) and `Faculty` (adds dutyCount, leaveDates).

`interface SeatingStrategy` → Implemented by `MixedSubjectSeating`, `ZigZagSeating`, `SequentialSeating`, `RandomSeating`.

4. PCCoE Pune Syllabus Alignment Matrix

Every concept listed in the S.Y. B.Tech Information Technology OOP curriculum is demonstrated in the code:

Syllabus Topic	Code Implementation & Exact Class Location
Default Constructor	`Person()`, `Student()`, `Faculty()`, `Course()`, `Room()`, `TimeSlot()` provide default values.
Parameterized Constructor	`Student(prn, name, age, div, sem, dept)` initializes fields directly upon instantiation.
Copy Constructor	`Person(Person other)`, `Student(Student other)`, `Course(Course other)` create safe deep copies.
Constructor Overloading	All entity classes feature multiple constructors with varying signatures in the same class.
'this' Keyword	Used for variable shadowing resolution (`this.name = name`) and constructor chaining (`this(...)`).
Encapsulation & Data Hiding	Member variables are declared `private` or `protected` with validated getters/setters.
Array of Objects	Used for room layout representation: `String[][] grid = new String[rows][cols]` in `SeatingService`.
Inheritance (extends)	Single & Hierarchical: `Student extends Person` and `Faculty extends Person`.
Access Modifiers	`protected` fields in `Person`, `private` fields in subclasses, and `public` service methods.
Abstract Classes	`Person` is an abstract base class with `public abstract void displayDetails();`.
Interfaces	`SeatingStrategy` defines the seating contract; `Comparable<Faculty>` orders the PriorityQueue.
Method Overriding	Subclasses override `displayDetails()`, `toString()`, `equals()`, `hashCode()`, and `compareTo()`.
Method Overloading	Overloaded constructors and helper search methods throughout the system.
Custom Exceptions	User-defined checked exceptions extending `Exception`: `SchedulingClashException`, `RoomCapacityExceededException`.
Exception Handling Keywords	Explicit use of `try`, `catch`, `finally`, `throw`, and `throws`.
Character Streams (File I/O)	`FileReader`, `BufferedReader`, `FileWriter`, `BufferedWriter` in `FileHandler.java`.

5. Algorithmic Logic & Mathematical Models

5.1 Greedy Graph Colouring for Clash-Free Timetabling

Let $G = (V, E)$ be an undirected conflict graph where each Course is a vertex $v \in V$. An edge $(u, v) \in E$ is added between Course u and Course v if there exists at least one student enrolled in both. A colour represents a distinct Time Slot.

Step-by-step trace of a 4-Course, 5-Student instance:

- Courses: C_1 (Java), C_2 (DSA), C_3 (DBMS), C_4 (Discrete Maths)
- Student S_1 takes C_1, C_2 → Edge (C_1, C_2)
- Student S_2 takes C_1, C_3 → Edge (C_1, C_3)
- Student S_3 takes C_2, C_3 → Edge (C_2, C_3)
- Students S_4, S_5 take C_4 → Disconnected from C_1, C_2, C_3

Course	Adjacent Neighbors in Graph	Neighbor Colours Used	Assigned Colour	Assigned Time Slot
C_1 (Java)	C_2, C_3	∅	**Colour 0**	Slot 1 (Day 1 Morning)
C_2 (DSA)	C_1, C_3	{0}	**Colour 1**	Slot 2 (Day 1 Afternoon)
C_3 (DBMS)	C_1, C_2	{0, 1}	**Colour 2**	Slot 3 (Day 2 Morning)
C_4 (Discrete)	∅ (no shared students)	∅	**Colour 0**	Slot 1 (Day 1 Morning)

Result: C_1 and C_4 run simultaneously in Slot 1 with **zero clashes**, needing only 3 total exam slots (chromatic number $\chi(G) = 3$).

5.2 Anti-Malpractice Interleaved Seating (Strategy Pattern)

When multiple exams occur in the same slot (e.g., C_1 and C_4), examinees are grouped into separate FIFO queues by course code. As the seating algorithm walks the room grid cell-by-cell (r, c), it pulls a student from an alternating queue. Consequently, direct physical neighbors (front, back, left, right) do not write the same subject paper, eliminating copying.

6. Key Code Chunks with Annotations

6.1 Abstraction and Constructor Chaining (`Person.java`)

```
public abstract class Person {
    protected String id;
    protected String name;
    protected int age;

    // Default constructor: calls parameterized constructor via this(...)
    public Person() {
        this("UNKNOWN", "Not Assigned", 18);
    }

    // Parameterized constructor: resolves name shadowing
    public Person(String id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    // Copy constructor: creates safe duplicate
    public Person(Person other) {
        if (other != null) {
            this.id = other.id;
            this.name = other.name;
            this.age = other.age;
        }
    }

    // Abstract method: mandatory implementation in subclasses
    public abstract void displayDetails();
}
```

6.2 Greedy Graph Colouring Logic (`TimetableGenerator.java`)

```
public List<Exam> generateSchedule() throws SchedulingClashException {
    buildConflictGraph(); // Draw edges between conflicting courses

    Map<Course, Integer> courseColors = new HashMap<>();

    for (Course currentCourse : courses) {
        Set<Integer> neighborColors = new HashSet<>();
        for (Course neighbor : conflictGraph.get(currentCourse)) {
            if (courseColors.containsKey(neighbor)) {
                neighborColors.add(courseColors.get(neighbor));
            }
        }

        // Find smallest non-conflicting color number
        int chosenColor = 0;
        while (neighborColors.contains(chosenColor)) {
            chosenColor++;
        }
        courseColors.put(currentCourse, chosenColor);
    }

    // Map color indices to real TimeSlot objects
    List<Exam> scheduledExams = new ArrayList<>();
    for (Course course : courses) {
        int colorIndex = courseColors.get(course);
        TimeSlot slot = availableSlots.get(colorIndex);
        scheduledExams.add(new Exam(course, slot));
    }
    return scheduledExams;
}
```

6.3 Character Stream CSV Export (`FileHandler.java`)

```
public static void exportTimetableToCSV(List<Exam> exams, String filePath) throws IOException {
    File file = new File(filePath);
    FileWriter fileWriter = null;
    BufferedWriter bufferedWriter = null;

    try {
        fileWriter = new FileWriter(file);
        bufferedWriter = new BufferedWriter(fileWriter);

        bufferedWriter.write("CourseCode,CourseName,Date,StartTime,EndTime,TotalStudents");
        bufferedWriter.newLine();

        for (Exam e : exams) {
            String line = String.format("%s,%s,%s,%s,%s,%d",
                    e.getCourse().getCode(), e.getCourse().getName(),
                    e.getTimeSlot().getDate(), e.getTimeSlot().getStartTime(),
                    e.getTimeSlot().getEndTime(), e.getEligibleStudents().size());
            bufferedWriter.write(line);
            bufferedWriter.newLine();
        }
        bufferedWriter.flush();
    } finally {
        // Safe resource cleanup in finally block
        if (bufferedWriter != null) bufferedWriter.close();
        if (fileWriter != null) fileWriter.close();
    }
}
```

7. Sample Output Snapshots

Generated Master Exam Timetable

```
================================================================================
                        MASTER EXAM TIMETABLE
================================================================================
DATE         | TIME SLOT    | COURSE     | SUBJECT NAME                  | STUDENTS
--------------------------------------------------------------------------------
2026-11-02   | 10:00 AM     | CS201      | Object Oriented Programming   | 5
2026-11-02   | 10:00 AM     | IT204      | Discrete Mathematics          | 5
2026-11-02   | 02:00 PM     | CS202      | Data Structures & Algorithms  | 5
2026-11-02   | 02:00 PM     | BS205      | Environmental Science         | 5
2026-11-03   | 10:00 AM     | IT203      | Database Management Systems   | 10
================================================================================
```

Generated Student Hall Ticket (PRN 101)

```
================================================================================
                   PCCoE PUNE - STUDENT HALL TICKET
================================================================================
Student Name : Aarav Sharma
PRN          : 101
Department   : IT (Div A)
--------------------------------------------------------------------------------
COURSE     | COURSE NAME               | DATE         | TIME         | ROOM         | SEAT
--------------------------------------------------------------------------------
CS201      | Object Oriented Programming | 2026-11-02   | 10:00 AM     | Auditorium   | R1-C2
CS202      | Data Structures & Algorithms| 2026-11-02   | 02:00 PM     | Auditorium   | R1-C1
IT203      | Database Management Systems | 2026-11-03   | 10:00 AM     | Auditorium   | R1-C1
================================================================================
```

Faculty Invigilation Duty Chart

```
================================================================================
                      FACULTY INVIGILATION DUTY CHART
================================================================================
FACULTY ID | NAME                | DATE         | TIME         | ROOM         | COURSE
--------------------------------------------------------------------------------
F001       | Dr. S. Sharma       | 2026-11-02   | 10:00 AM     | Auditorium   | CS201
F004       | Dr. A. Deshmukh     | 2026-11-02   | 10:00 AM     | Auditorium   | IT204
F002       | Prof. P. Kulkarni   | 2026-11-02   | 02:00 PM     | Auditorium   | CS202
F003       | Prof. R. Patil      | 2026-11-02   | 02:00 PM     | Auditorium   | BS205
F001       | Dr. S. Sharma       | 2026-11-03   | 10:00 AM     | Auditorium   | IT203
--------------------------------------------------------------------------------
DUTY SUMMARY (Fairness Distribution):
 -> Dr. S. Sharma (F001)     - Total Duties: 2
 -> Prof. P. Kulkarni (F002) - Total Duties: 1
 -> Prof. R. Patil (F003)    - Total Duties: 1
 -> Dr. A. Deshmukh (F004)   - Total Duties: 1
================================================================================
```

8. Teacher Viva Preparation & Top Questions

Q1: Why is Person an abstract class?

Because a generic person cannot sit for an exam or invigilate; only a specific Student or Faculty can. Person serves as an abstract base class providing common attributes and enforcing the displayDetails() method.

Q2: How does Graph Colouring prevent scheduling clashes?

An edge connects two courses if any student takes both. Because graph colouring guarantees adjacent vertices never receive the same colour, courses sharing students are never assigned the same time slot.

Q3: Why did you use the Strategy Design Pattern?

To allow swapping seating algorithms (MixedSubject, ZigZag, Sequential, Random) dynamically at runtime without modifying the SeatingService class.

Q4: How does PriorityQueue ensure fairness in invigilation duties?

Faculty implements Comparable<Faculty> based on dutyCount. The min-heap always pulls the teacher with the fewest duties assigned so far.

Q5: Why choose Character Streams over Byte Streams?

Character streams (FileReader, BufferedReader, BufferedWriter) work directly with 16-bit Unicode characters, making them optimal for text and CSV files.

9. Conclusion

The **Exam Timetable & Seating Generator** mini project successfully proves how fundamental Object-Oriented Programming (OOP) concepts, design patterns, graph theory, and character stream file handling can be synthesized into an effective, real-world college administrative solution. The code is modular, self-explanatory, beginner-friendly, and completely aligned with the second-year Information Technology curriculum at PCCoE Pune.